

RAG Chatbot

8-Day Build Roadmap — Engineered for AI Engineer Interviews

PyPDF2/pypdf · sentence-transformers · ChromaDB · Groq API · Streamlit

This is Project 2 in the portfolio sequence (after the Geopolitical Threat Intelligence Analytics Pipeline). Each day below keeps the original data-science-oriented plan intact, and adds an **AI Engineer layer** — production-readiness, evaluation rigor, and system-design thinking — the things interviewers for AI Engineer roles specifically probe for beyond "does the demo work."

- Core task (DS/tutorial baseline)
- AI Engineer addition

DAY 1 Setup & Document Ingestion

CORE TASKS

- Set up environment: **pip install chromadb sentence-transformers pypdf streamlit groq**
- Get a free Groq API key (groq.com)
- Write a script to load a PDF/docx and extract raw text
- Test on 2-3 sample documents (research paper, handbook, textbook chapter)

AI ENGINEER LAYER (add this)

- Wrap the Groq client setup in a config module (API key, model name, timeouts) — no hardcoded secrets
- Add basic logging (timestamps + doc name + extraction time) instead of print statements

DELIVERABLE → Working text extraction script

DAY 2 Chunking Strategy

CORE TASKS

- Implement chunking (LangChain's RecursiveCharacterTextSplitter or manual sliding window)
- Experiment with chunk size (e.g. 500 chars, 100 overlap) — print and inspect chunks manually
- Store chunk metadata: {text, source_doc, page_number, chunk_id}

AI ENGINEER LAYER (add this)

- Make chunk size / overlap configurable parameters, not hardcoded — you'll sweep these in Day 7's eval
- Note token count per chunk (not just char count) — this maps directly to LLM context-window cost

DELIVERABLE → Clean list of chunks with metadata, ready to embed

DAY 3

Embeddings & Vector Store

CORE TASKS

- Generate embeddings using sentence-transformers
- Set up ChromaDB, create a collection, insert chunks with embeddings + metadata
- Write a basic similarity search function: given a query, return top-k chunks
- Test manually: ask a question, check if retrieved chunks make sense

AI ENGINEER LAYER (add this)

- Log embedding throughput (chunks/sec) — this is the number you'd quote if asked "how would this scale to 10K docs?"
- Batch the embedding calls rather than looping one chunk at a time

DELIVERABLE → Working ChromaDB with searchable embeddings

DAY 4

Retrieval Pipeline + Prompt Construction

CORE TASKS

- Build the full retrieval function: query → embed → search → top-k chunks
- Construct the LLM prompt: system instruction + retrieved context + user question
- Call the Groq API with this prompt, get a grounded answer
- Add source citation: track which chunk/doc/page the answer came from, display alongside response

AI ENGINEER LAYER (add this)

- Add retry logic with exponential backoff around the Groq API call (handle rate limits / transient failures)
- Track and log latency separately for embed time, retrieval time, and LLM response time
- Add a config flag for top-k so it's easy to sweep in evaluation

DELIVERABLE → End-to-end script: ask a question in terminal → get a grounded, cited answer

DAY 5

Add Chat Memory + Multi-turn

CORE TASKS

- Implement conversation history (store last N turns, include in prompt for context)
- Handle follow-up questions (e.g. "what about its limitations?")
- Test with a multi-turn conversation to confirm context carries over

AI ENGINEER LAYER (add this)

- Track token usage running total across a conversation — this is what production cost monitoring looks like
- Cache identical repeated queries in-session (simple dict cache) — a cheap, real optimization to mention

DELIVERABLE → RAG pipeline supporting multi-turn chat

DAY 6 Build Streamlit Chatbot UI

CORE TASKS

- Streamlit chat interface (st.chat_message, st.chat_input)
- File uploader so a user can upload their own PDF and chat with it
- Display retrieved sources/citations alongside each answer (expandable section)
- Loading states, error handling (no document uploaded, API failure)

AI ENGINEER LAYER (add this)

- Surface latency + token count per response in the UI (small caption under each answer) — makes cost/perf visible, not hidden
- Handle the Groq API failure path gracefully in the UI (fallback message, not a stack trace)

DELIVERABLE → Working chatbot web app

DAY 7 Evaluation + Polish

CORE TASKS

- Create a small test set: 10 questions with known answers from your test document
- Measure: does retrieval pull the correct chunk? (manual check, simple % accuracy)
- Try 2 different chunk sizes, compare retrieval accuracy — document the finding

AI ENGINEER LAYER (add this)

- Add a faithfulness check: does the generated answer stay grounded in the retrieved chunks, or does it add unsupported claims?
- Build a small latency vs. accuracy tradeoff table across top-k = 3 / 5 and chunk size = 300 / 500
- Write down what you'd optimize next and why — this becomes your "what would you improve" interview answer

DELIVERABLE → A short evaluation report with a retrieval-accuracy table and a latency/accuracy tradeoff table

CORE TASKS

- Write README: architecture diagram, design decisions (why ChromaDB, why this chunk size, why a manual pipeline), evaluation results, limitations
- Clean code, push to GitHub
- Buffer for bugs; rehearse a 5-minute demo (upload doc → ask question → show cited answer → show follow-up working)

AI ENGINEER LAYER (add this)

- Write a short 'Scaling This System' section: what breaks first at 10K docs / 1000 concurrent users (vector DB throughput, embedding throughput, LLM rate limits) and what you'd change (managed vector DB, async processing, response caching)
- Prepare 3 interview-ready stories: a tradeoff you made and why, a bug you debugged, one thing you'd do differently at scale

DELIVERABLE → Polished GitHub repo + a written scaling narrative, ready to demo and defend in an interview

Resume Bullet (after completion)

“Built a Retrieval-Augmented Generation chatbot from scratch (no LangChain abstraction layer) — implemented custom chunking, embedding (sentence-transformers), vector retrieval (ChromaDB), and multi-turn conversational memory; evaluated retrieval accuracy and answer faithfulness across chunking strategies, tracked latency/cost per query, and deployed via Streamlit with source-cited responses.”